

**LAPORAN PRAKTIKUM**  
**ALGORITMA PEMROGRAMAN 2**  
**MODUL 2 – REVIEW PENGENALAN PEMROGRAMAN**



NAMA : AIS NURHANIFAH

NIM : 109082500162

KELAS : S1IF-13-06

**PROGRAM STUDI TEKNIK INFORMATIKA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**

**2026**

## A. Dasar Teori

Bahasa pemrograman Go (Golang) menyediakan dasar yang kuat untuk bikin program komputer lewat tiga konsep logika utama. Konsep pertama yaitu struktur runtunan (sekuensial) dan cara atur variabel. Di teori dasarnya, variabel itu dipakai buat tempat simpan data sementara di memori komputer. Nah, tipenya harus pas; kalau angka desimal yang butuh ketelitian tinggi kita pakai float64, tapi kalau angka bulat cukup pakai int. Variabel-variabel ini nantinya bakal dihitung pakai operasi matematika biasa kaya kali (\*), bagi (/), dan modulus (%) buat ngubah input mentah jadi data yang siap dipakai.

Konsep kedua yaitu percabangan atau kondisional, yang bikin program bisa ngambil keputusan sendiri. Secara teori, perintah kaya if, else if, dan else bakal ngecek suatu syarat tertentu. Kalau syaratnya cocok (bernilai *true*), kode di dalamnya baru jalan. Aturan percabangan ini bisa sesederhana nyaring kategori umur penonton tiket, ngecek tahun kabisat pakai bantuan operator logika (|| dan &&), sampai ke percabangan berlapis (*nested if*). Percabangan berlapis ini penting banget kalau kita punya skenario yang ribet, di mana keputusan akhir baru bisa diambil setelah lolos beberapa kali seleksi kondisi yang bertingkat.

Konsep ketiga yaitu perulangan (*looping*), sebuah cara biar program bisa jalanin perintah yang sama berkali-kali tanpa kita harus nulis kodenya berulang-ulang. Di bahasa Go, semua urusan perulangan ini diatur pakai perintah for. Secara konsep, perulangan ini dibagi jadi dua macam berdasarkan jumlah jalannya proses. Yang pertama ada perulangan pakai kondisi, jadi program bakal terus muter selama syaratnya masih terpenuhi dan baru berhenti pas udah mentok sama target. Yang kedua ada perulangan yang jumlahnya udah pasti dari awal (*counted loop*), yang pas banget dipakai buat ngecek atau nguji data dalam batas urutan tertentu.

## B. Guided

### 1. [boddymassindex.go]

```
package main
import "fmt"

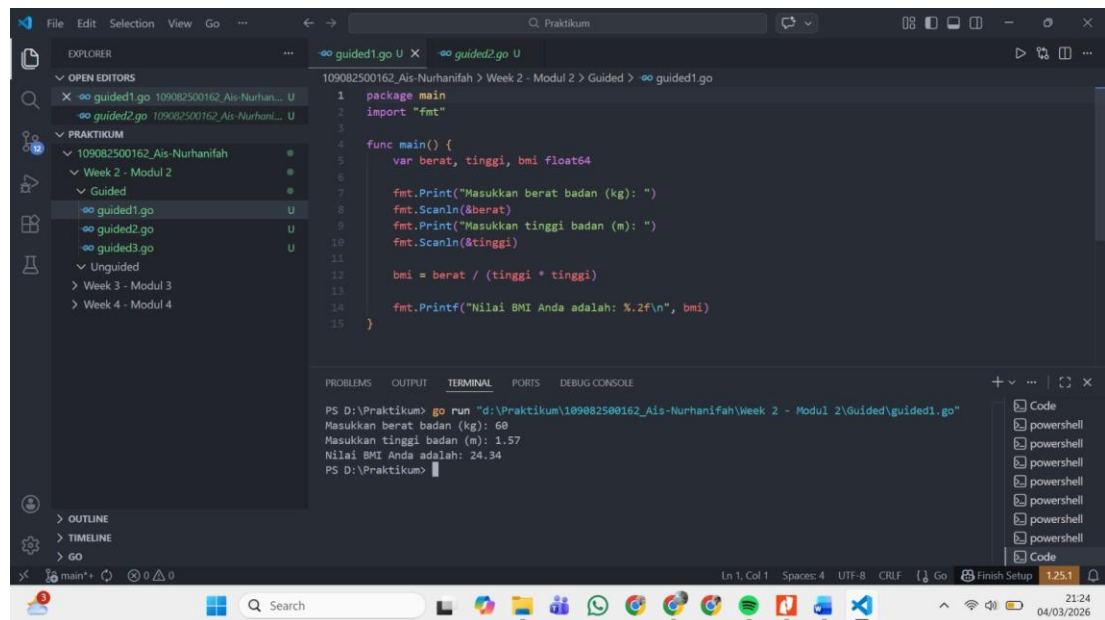
func main() {
    var berat, tinggi, bmi float64

    fmt.Print("Masukkan berat badan (kg): ")
    fmt.Scanln(&berat)
    fmt.Print("Masukkan tinggi badan (m): ")
    fmt.Scanln(&tinggi)

    bmi = berat / (tinggi * tinggi)

    fmt.Printf("Nilai BMI Anda adalah: %.2f\n", bmi)
}
```

Output :



```
1 package main
2 import "fmt"
3
4 func main() {
5     var berat, tinggi, bmi float64
6
7     fmt.Print("Masukkan berat badan (kg): ")
8     fmt.Scanln(&berat)
9     fmt.Print("Masukkan tinggi badan (m): ")
10    fmt.Scanln(&tinggi)
11
12    bmi = berat / (tinggi * tinggi)
13
14    fmt.Printf("Nilai BMI Anda adalah: %.2f\n", bmi)
15 }
```

PS D:\Praktikum> go run "d:\Praktikum\109082500162\_Ais-Nurhanifah\Week 2 - Modul 2\Guided\guided1.go"

Masukkan berat badan (kg): 60

Masukkan tinggi badan (m): 1.57

Nilai BMI Anda adalah: 24.34

PS D:\Praktikum>

Penjelasan:

Program ini adalah aplikasi berbasis bahasa pemrograman Go (Golang) yang dibuat untuk menghitung Indeks Massa Tubuh (BMI) secara otomatis. Kode ini diawali dengan menyiapkan tiga variabel bertipe data desimal (float64), yaitu berat, tinggi, dan bmi. Program kemudian berinteraksi dengan pengguna menggunakan fungsi input-output untuk meminta input berat badan dalam satuan kilogram dan tinggi badan dalam satuan meter, lalu menyimpannya ke dalam memori komputer.

Setelah data tersimpan, program melakukan kalkulasi dengan rumus standar, yakni membagi berat badan dengan hasil kuadrat tinggi badan. Hasil akhir dari perhitungan tersebut kemudian dicetak menggunakan format khusus yang membatasi tampilan angka agar hanya memuat dua digit di belakang koma. Melalui proses ini, pengguna bisa langsung mengetahui hasil akhir perhitungan dengan tampilan yang lebih rapi dan presisi, seperti nilai 24.34 yang dihasilkan pada contoh data tersebut.

## 2. [targetTabungan.go]

```
package main
import "fmt"

func main() {
    var target, tabungan, total, hari int
```

```

    fmt.Print("Masukkan target uang yang ingin dicapai: ")
    fmt.Scanln(&target)

    total = 0
    hari = 0

    for total < target {
        hari++
        fmt.Printf("Masukkan nominal tabungan hari ke-%d: ",
hari)
        fmt.Scanln(&tabungan)

        total = total + tabungan
    }

    fmt.Printf("Selamat! Target tercapai dalam %d hari.\n",
hari)
    fmt.Printf("Total tabungan Anda terkumpul: Rp%d\n",
total)
}

```

Output :

The screenshot shows a Go IDE with the following components:

- Explorer:** Shows the project structure with folders for 'Week 2 - Modul 2', 'Guided', and 'Unguided'. The 'Guided' folder contains 'guided1.go', 'guided2.go', and 'guided3.go'.
- Editor:** Displays the source code for 'guided2.go', which matches the code provided in the first block.
- Terminal:** Shows the execution of the program using the command `go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Guided\guided2.go"`. The output shows the user entering a target of 1000000 and daily savings of 250000 for 4 days, resulting in a total of 1000000.

Penjelasan:

Program ini adalah simulasi pencatatan tabungan harian untuk mencapai suatu target finansial tertentu. Pada awal proses, kode ini menyiapkan empat variabel bertipe data bilangan bulat (int), yaitu target, tabungan, total, dan hari, serta menginisialisasi nilai awal total tabungan dan jumlah hari dari angka nol. Setelah itu, program meminta pengguna untuk memasukkan nominal target uang yang ingin dicapai lalu menyimpannya ke dalam sistem.

Proses inti dari program ini terletak pada mekanisme perulangan kondisi (for total < target). Selama akumulasi tabungan belum menyentuh atau melewati angka target, program akan terus menambah hitungan hari dan meminta pengguna memasukkan nominal tabungan harian yang baru secara berulang. Setiap kali nominal baru dimasukkan, nilainya langsung ditambahkan ke variabel total. Perulangan akan otomatis berhenti begitu target terpenuhi, lalu program menampilkan teks ucapan selamat yang menginformasikan jumlah hari yang dihabiskan serta total saldo akhir yang berhasil dikumpulkan.

### 3. [KategoriTiket.go]

```
package main
import "fmt"

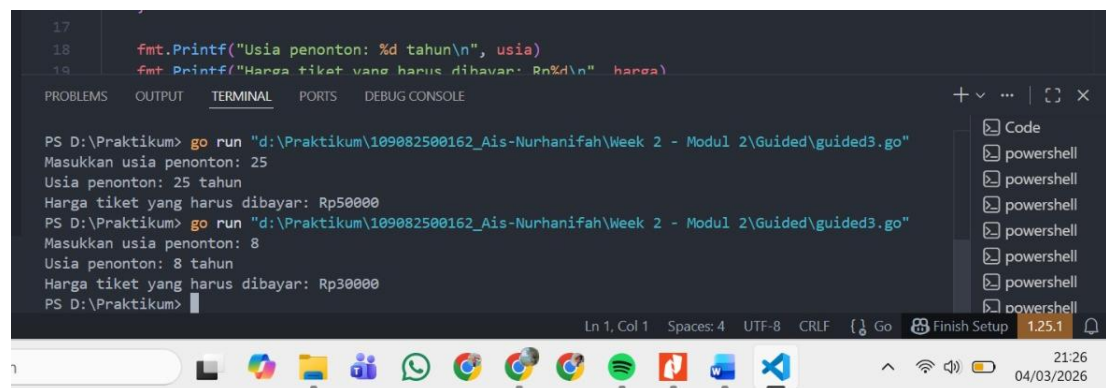
func main() {
    var usia, harga int

    fmt.Print("Masukkan usia penonton: ")
    fmt.Scanln(&usia)

    if usia < 12 {
        harga = 30000
    } else if usia >= 12 && usia <= 17 {
        harga = 40000
    } else {
        harga = 50000
    }

    fmt.Printf("Usia penonton: %d tahun\n", usia)
    fmt.Printf("Harga tiket yang harus dibayar: Rp%d\n",
harga)
}
```

Output :



```
17
18     fmt.Printf("Usia penonton: %d tahun\n", usia)
19     fmt.Printf("Harga tiket yang harus dibayar: Rp%d\n", harga)

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE

PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Guided\guided3.go"
Masukkan usia penonton: 25
Usia penonton: 25 tahun
Harga tiket yang harus dibayar: Rp50000
PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Guided\guided3.go"
Masukkan usia penonton: 8
Usia penonton: 8 tahun
Harga tiket yang harus dibayar: Rp30000
PS D:\Praktikum>
```

Penjelasan:

Program ini adalah sistem penentu harga tiket nonton otomatis yang disesuaikan berdasarkan kategori umur seseorang. Pada bagian awal, kode menyediakan dua variabel bertipe data bilangan bulat (int), yaitu usia untuk mencatat umur yang dimasukkan oleh pengguna dan harga untuk menampung nominal tarif tiket yang nantinya harus dibayar setelah melalui proses pengecekan.

Proses seleksi tarif dilakukan menggunakan struktur logika percabangan kondisional (if-else). Jika umur yang dimasukkan berada di bawah 12 tahun, tarif ditetapkan sebesar Rp30.000, sedangkan untuk rentang umur 12 hingga 17 tahun dikenakan tarif Rp40.000, dan di luar kondisi tersebut secara otomatis dianggap sebagai kategori dewasa dengan tarif Rp50.000. Setelah kategori yang cocok ditemukan, sistem akan langsung memproses dan menampilkan rangkuman data berupa informasi umur penonton beserta nominal tarif akhir yang wajib dibayarkan.

### C. Unguided

#### 1. Soal Unguided 1

Tahun kabisat adalah tahun yang habis dibagi 400 atau habis dibagi 4 tetapi tidak habis dibagi 100. Buatlah sebuah program yang menerima input sebuah bilangan bulat dan memeriksa apakah bilangan tersebut merupakan tahun kabisat (true) atau bukan (false). Lakukan pengecekan apakah tahun 2000, 1974, 2106, 1876 merupakan tahun kabisat atau bukan!

Jawaban :

```
package main

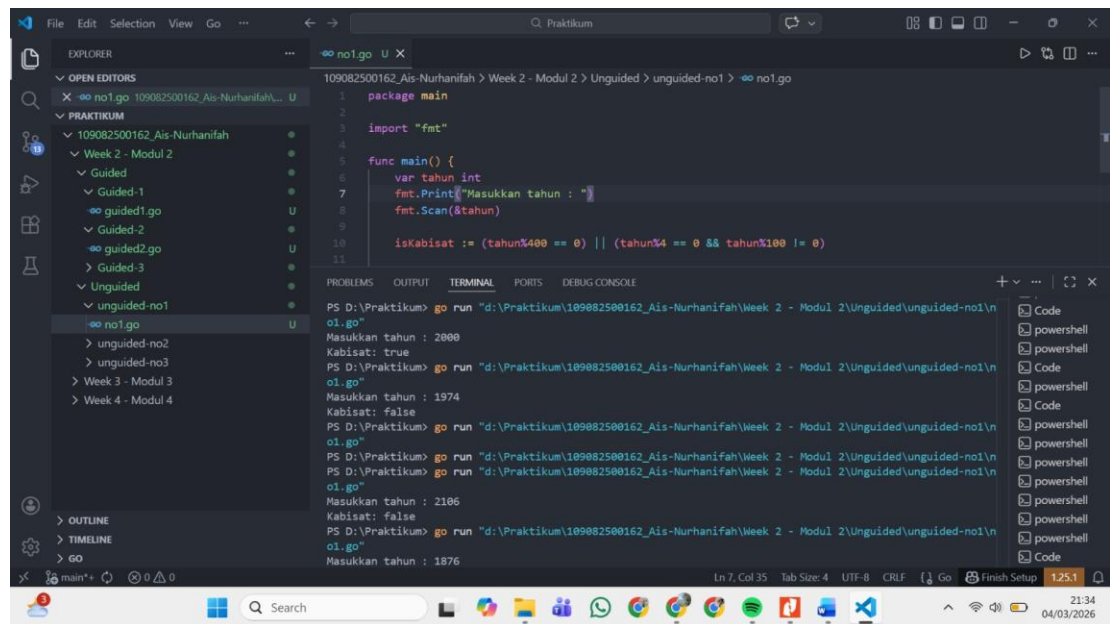
import "fmt"

func main() {
    var tahun int
    fmt.Print("Masukkan tahun : ")
    fmt.Scan(&tahun)

    isKabisat := (tahun%400 == 0) || (tahun%4 == 0 &&
    tahun%100 != 0)

    fmt.Println("Kabisat:", isKabisat)
}
```

Output :



```
109082500162_Ais-Nurhanifah > Week 2 - Modul 2 > Unguided > unguided-no1 > no1.go
1 package main
2
3 import "fmt"
4
5 func main() {
6     var tahun int
7     fmt.Print("Masukkan tahun : ")
8     fmt.Scan(&tahun)
9
10    isKabisat := (tahun%400 == 0) || (tahun%4 == 0 && tahun%100 != 0)
11
12    // Output:
13    // PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Unguided\unguided-no1\n
14    // o1.go"
15    // Masukkan tahun : 2000
16    // Kabisat: true
17    // PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Unguided\unguided-no1\n
18    // o1.go"
19    // Masukkan tahun : 1974
20    // Kabisat: false
21    // PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Unguided\unguided-no1\n
22    // o1.go"
23    // Masukkan tahun : 2106
24    // Kabisat: false
25    // PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Unguided\unguided-no1\n
26    // o1.go"
27    // Masukkan tahun : 1876
```

Penjelasan :

Program ini adalah sistem validasi otomatis yang dirancang untuk menguji konsistensi urutan warna dalam lima kali percobaan eksperimen. Di awal proses, kode ini menginisialisasi sebuah variabel boolean bernama berhasil dengan nilai awal true, serta menyiapkan empat variabel teks (string) yaitu w1 hingga w4 untuk menampung empat input warna di setiap sesi. Program kemudian masuk ke dalam mekanisme perulangan tetap (for) yang akan berjalan tepat sebanyak lima kali.

Di setiap putaran perulangan, pengguna diminta memasukkan kombinasi empat warna sekaligus untuk diperiksa oleh sistem menggunakan logika percabangan. Jika ada satu saja warna yang tidak sesuai dengan urutan baku—yaitu merah, kuning, hijau, dan ungu—maka kondisi operator || (atau) akan terpenuhi dan mengubah status variabel berhasil menjadi false. Setelah seluruh rangkaian lima percobaan selesai dievaluasi, sistem akan menampilkan status final berupa true jika semua urutan di setiap percobaan tepat, atau false jika terdapat kesalahan pada salah satu sesi pengisian.

## 2. Soal Unguided 2

Siswa kelas IPA di salah satu sekolah menengah atas di Indonesia sedang mengadakan praktikum kimia. Di setiap percobaan akan menggunakan 4 tabung reaksi, yang mana susunan warna cairan di setiap tabung akan menentukan hasil percobaan. Siswa diminta untuk mencatat hasil percobaan tersebut. Percobaan

dikatakan berhasil apabila susunan warna zat cair pada gelas 1 hingga gelas 4 secara berturutan adalah 'merah', 'kuning', 'hijau', dan 'ungu' selama 5 kali percobaan berulang. Buatlah sebuah program yang menerima input berupa warna dari ke 4 gelas reaksi sebanyak 5 kali percobaan. Kemudian program akan menampilkan true apabila urutan warna sesuai dengan informasi yang diberikan pada paragraf sebelumnya, dan false untuk urutan warna lainnya.

Jawaban :

```
package main

import "fmt"

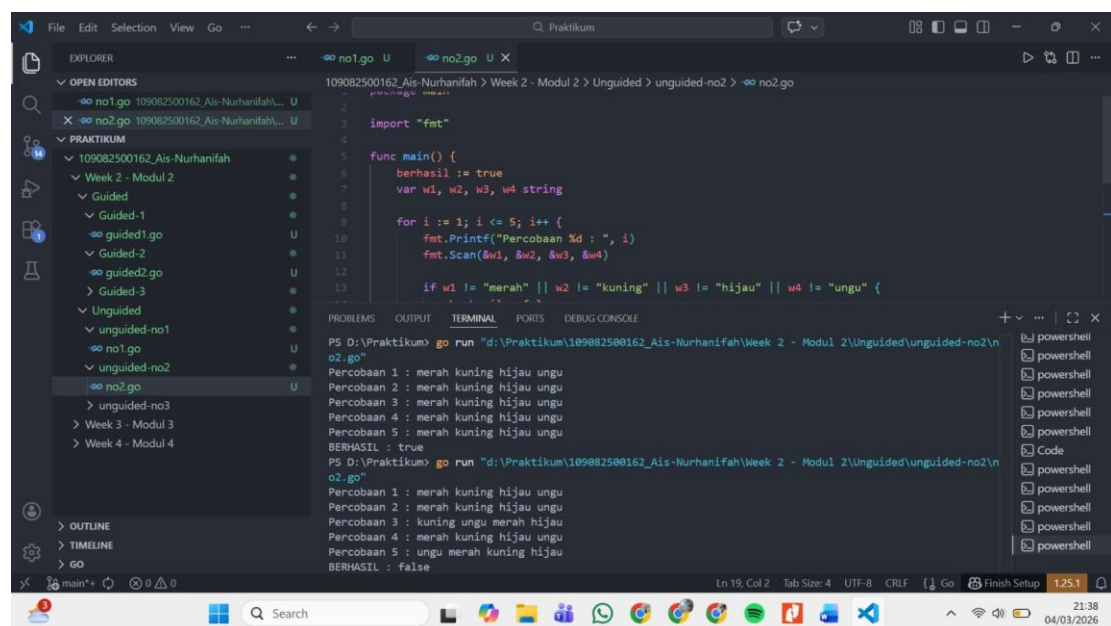
func main() {
    berhasil := true
    var w1, w2, w3, w4 string

    for i := 1; i <= 5; i++ {
        fmt.Printf("Percobaan %d : ", i)
        fmt.Scan(&w1, &w2, &w3, &w4)

        if w1 != "merah" || w2 != "kuning" || w3 !=
"hijau" || w4 != "ungu" {
            berhasil = false
        }
    }

    fmt.Println("BERHASIL :", berhasil)
}
```

Output :



The screenshot shows the Visual Studio Code interface with a Go file named `no2.go` open. The code in the editor matches the provided solution. The terminal at the bottom shows the execution of the program using `go run`. The output displays five iterations of input and the final result.

```
PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Unguided\unguided-no2\n
o2.go"
Percobaan 1 : merah kuning hijau ungu
Percobaan 2 : merah kuning hijau ungu
Percobaan 3 : merah kuning hijau ungu
Percobaan 4 : merah kuning hijau ungu
Percobaan 5 : merah kuning hijau ungu
BERHASIL : true
PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Unguided\unguided-no2\n
o2.go"
Percobaan 1 : merah kuning hijau ungu
Percobaan 2 : merah kuning hijau ungu
Percobaan 3 : kuning ungu merah hijau
Percobaan 4 : merah kuning hijau ungu
Percobaan 5 : ungu merah kuning hijau
BERHASIL : false
```



Penjelasan :

Program ini adalah sistem validasi otomatis yang dirancang untuk menguji konsistensi urutan warna dalam lima kali percobaan eksperimen. Di awal proses, kode ini menginisialisasi sebuah variabel boolean bernama berhasil dengan nilai awal true, serta menyiapkan empat variabel teks (string) yaitu w1 hingga w4 untuk menampung empat input warna di setiap sesi. Program kemudian masuk ke dalam mekanisme perulangan tetap (for) yang akan berjalan tepat sebanyak lima kali.

Di setiap putaran perulangan, pengguna diminta memasukkan kombinasi empat warna sekaligus untuk diperiksa oleh sistem menggunakan logika percabangan. Jika ada satu saja warna yang tidak sesuai dengan urutan baku—yaitu merah, kuning, hijau, dan ungu—maka kondisi operator || (atau) akan terpenuhi dan mengubah status variabel berhasil menjadi false. Setelah seluruh rangkaian lima percobaan selesai dievaluasi, sistem akan menampilkan status final berupa true jika semua urutan di setiap percobaan tepat, atau false jika terdapat kesalahan pada salah satu sesi pengisian.

### 3. Soal Unguided 3

PT POS membutuhkan aplikasi perhitungan biaya kirim berdasarkan berat parcel. Maka, buatlah program BiayaPos untuk menghitung biaya pengiriman tersebut dengan ketentuan sebagai berikut! Dari berat parcel (dalam gram), harus dihitung total berat dalam kg dan sisanya (dalam gram). Biaya jasa pengiriman adalah Rp. 10.000,- per kg. Jika sisa berat lebih dari 500 gram, maka tambahan biaya kirim hanya Rp. 5,- per gram saja. Tetapi jika sisa berat kurang dari 500 gram, maka tambahan biaya akan dibebankan sebesar Rp. 15,- per gram. Sisa berat (yang kurang dari 1kg) digratiskan biayanya apabila total berat ternyata lebih dari 10kg. Buatlah detail perhitungan yang meliputi detail berat, detail biaya, dan total biaya apabila berat parcel yang dikirim adalah 8500 gram, 9250 gram, dan 11750 gram.

Jawaban :

```
package main

import "fmt"

func main() {
    var totalGram int
    fmt.Print("Masukkan total berat (gram): ")
    fmt.Scan(&totalGram)

    kg, gr := totalGram/1000, totalGram%1000
    biayaKg := kg * 10000
    biayaGr := 0
```

```

        if kg < 10 {
            if gr >= 500 {
                biayaGr = gr * 5
            } else {
                biayaGr = gr * 15
            }
        }

        fmt.Println("==== Detail Perhitungan =====")
        fmt.Printf("Detail berat : %d kg + %d gram\n", kg,
gr)
        fmt.Printf("Detail biaya : Rp. %d + Rp. %d\n",
biayaKg, biayaGr)
        fmt.Printf("Total biaya: Rp %d\n", biayaKg +
biayaGr)
    }
}

```

Output :

```

PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Unguided\unguided-no3\
o3.go"
Masukkan total berat (gram): 8500
==== Detail Perhitungan ====
Detail berat : 8 kg + 500 gram
Detail biaya : Rp. 80000 + Rp. 2500
Total biaya: Rp 82500
PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Unguided\unguided-no3\
o3.go"
Masukkan total berat (gram): 9250
==== Detail Perhitungan ====
Detail berat : 9 kg + 250 gram
Detail biaya : Rp. 90000 + Rp. 3750
Total biaya: Rp 93750
PS D:\Praktikum> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 2 - Modul 2\Unguided\unguided-no3\
o3.go"
Masukkan total berat (gram): 11750
==== Detail Perhitungan ====
Detail berat : 11 kg + 750 gram
Detail biaya : Rp. 110000 + Rp. 0
Total biaya: Rp 110000
PS D:\Praktikum>

```

Penjelasan :

Program ini adalah sistem kalkulator logistik yang digunakan untuk menghitung rincian biaya pengiriman barang berdasarkan konversi berat dari satuan gram ke kilogram. Pada awal proses, sistem meminta pengguna memasukkan total berat barang dalam satuan gram ke dalam variabel totalGram. Angka tersebut kemudian langsung dipecah menjadi dua komponen menggunakan operasi pembagian untuk mendapatkan nilai kilogram (kg) dan operasi modulus atau sisa pembagian untuk mendapatkan sisa satuan gram (gr). Biaya dasar pengiriman ditetapkan sebesar Rp10.000 per kilogram.

Proses penentuan biaya tambahan untuk sisa gram diatur melalui struktur percabangan berlapis (if bersarang). Apabila total berat barang masih di bawah 10 kilogram, sistem akan memeriksa sisa gramnya; jika sisa tersebut mencapai 500 gram atau lebih, sisa gram akan dikalikan dengan tarif Rp5 per gram, sedangkan jika di bawah 500 gram, tarifnya menjadi Rp15 per gram. Namun, jika total berat barang sudah mencapai atau melebihi 10 kilogram, kondisi pengecekan ini dilewati sehingga biaya tambahan sisa gram secara otomatis dianggap gratis atau nol rupiah, seperti yang terlihat pada pengujian data 11.750 gram. Di akhir proses, seluruh rincian berat, komponen biaya, serta akumulasi total biaya dicetak secara terstruktur.

#### D. Kesimpulan

Berdasarkan seluruh program yang telah diuji, dapat disimpulkan bahwa pemrograman dasar tidak hanya berfokus pada bagaimana cara menyimpan data ke dalam variabel, melainkan bagaimana mengendalikan alur data tersebut secara logis dan efisien. Penggunaan tipe data yang tepat, seperti bilangan bulat (int) untuk perhitungan hari, tarif tiket, atau tahun, serta desimal (float64) untuk data yang membutuhkan akurasi tinggi seperti indeks massa tubuh, sangat krusial dalam mencegah terjadinya kesalahan fatal atau kehilangan presisi nilai saat kalkulasi berlangsung. Pemilihan operator aritmatika dan operator sisa pembagian (modulus) juga terbukti menjadi instrumen penting dalam memecah atau mengonversi satuan data mentah menjadi informasi yang lebih spesifik.

Selain itu, kombinasi antara struktur percabangan (if-else) dan perulangan (for) menjadi kunci utama dalam membangun sistem yang interaktif dan adaptif terhadap berbagai kondisi dunia nyata. Mekanisme perulangan, baik yang berbasis kondisi tertentu maupun pencacah tetap, sangat mempermudah otomatisasi tugas secara berulang tanpa redundansi kode. Sementara itu, logika percabangan tunggal hingga percabangan bersarang (*nested if*) memungkinkan sistem mengambil keputusan berlapis yang kompleks secara terstruktur, seperti pada penentuan tarif logistik yang bergantung pada batas kuota berat barang. Keseluruhan konsep logika ini merupakan fondasi utama yang saling melengkapi dalam menyelesaikan berbagai masalah komputasi secara sistematis.

